SCX 框架工程实现方案

State-Conditioned eXpertise — 面向数据价值评估与专家引导学习的状态条件专家性框架版本: v0.1.0 | 最后更新: 2026-06-25

目录

---

1. Python 实现架构设计

1.1 包目录结构总览

```
src/scx/
   __init__.py                 # 版本号、顶级 API 导出
   core/
      __init__.py
      framework.py             # SCXFramework — 顶层统一接口
      config.py                # SCXConfig — 配置管理与校验
      metrics.py               # SCXMetrics — 评估指标
```

```
state/
    __init__.py
    space.py              # StateSpace — 状态空间管理
    discovery.py          # StateDiscovery — 状态发现算法
    assignment.py         # StateAssignment — 硬/软状态分配
    metrics.py            # StateMetrics — 状态质量评估
expert/
    __init__.py
    registry.py           # ExpertRegistry — 专家注册与管理
    reliability.py        # ExpertReliability — 可靠性估计
    router.py             # ExpertRouter — 硬路由与软加权
    conflict.py           # ExpertConflict — 冲突检测与仲裁
valuation/
    __init__.py
    classifier.py         # DataClassifier — 数据四分类
    state_value.py        # StateValue — V(s) 计算
    learnability.py       # LearnabilityScore — L(s) 估计
    noise_score.py        # NoiseScore — N(x) 估计
    redundancy.py         # RedundancyScore — D(s) 估计
action/
    __init__.py
    policy.py             # ActionPolicy — 动作决策引擎
    acquisition.py        # AcquisitionStrategy — 采集策略
    compress.py           # CompressStrategy — 冗余压缩策略
utils/
    __init__.py
    visualization.py      # 可视化工具 (plotly + matplotlib)
    data_loader.py        # 数据加载工具
    helpers.py            # 通用辅助函数
```

```
    evaluation.py          # 评估辅助工具
```

1.2 模块详细设计

---

scx/state/ — 状态空间模块　scx/state/space.py — StateSpace

状态空间的容器与查询接口，是 SCX 框架的核心数据结构。

```python
@dataclass
class StateInfo:
    """ 单个状态的元数据"""
    id: int                          # 状态编号
    label: str                       # 可读标签（可选）
    centroid: np.ndarray             # 质心坐标   空间
    radius: float                    # 半径（类内标准差）
    count: int                       # 样本计数
    proportion: float                #  (s) = count / total

class StateSpace:
    """ 管理状态空间 S = {s_1, ..., s_K}"""

    def __init__(self, n_states: int):
        self.n_states = n_states
        self.states: dict[int, StateInfo] = {}
        self.assignment: np.ndarray | None = None        # (N,) 硬分配标签
        self.soft_assignment: np.ndarray | None = None   # (N, K) 软分配矩阵
```

```
    def add_state(self, state: StateInfo) -> None
    def get_state(self, state_id: int) -> StateInfo
    def remove_state(self, state_id: int) -> None
    def get_centroids(self) -> np.ndarray                # shape (K, d)
    def get_proportions(self) -> np.ndarray              # shape (K,)
    def summary(self) -> pd.DataFrame                    # 状态汇总表
    def save(self, path: str) -> None                    # 序列化
    @classmethod
    def load(cls, path: str) -> StateSpace
```

依赖：numpy, pandas, pickle（序列化）

---

scx/state/discovery.py — StateDiscovery

从 (X) 中自动发现状态结构。

```
class StateDiscovery:
    """ 从表示空间 (X) 发现状态（聚类/降维 + 聚类）"""

    def __init__(self, method: str = "kmeans", n_states: int = 10,
                 random_state: int = 42):
        """
        Parameters
        ----------
        method : str
            支持 'kmeans' | 'gmm' | 'spectral' | 'hdbscan' | 'vae'
        n_states : int
            目标状态数（HDBSCAN 自动确定）
        """
```

```
        self.method = method
        self.n_states = n_states
        self.model = None

    def fit(self, X_phi: np.ndarray) -> "StateDiscovery":
        """ 在 (X) 上拟合状态发现算法

        Parameters
        ----------
        X_phi : np.ndarray, shape (N, d)
            表示空间中的特征矩阵
        """
        ...

    def predict(self, X_phi: np.ndarray) -> np.ndarray:
        """ 为新的 (x) 分配状态 ID"""
        ...

    def fit_predict(self, X_phi: np.ndarray) -> np.ndarray:
        """ 拟合并返回标签"""
        ...

    def get_centroids(self) -> np.ndarray:
        """ 返回状态质心, shape (K, d)"""
        ...

    def get_probabilities(self, X_phi: np.ndarray) -> np.ndarray:
        """ 返回软分配概率, shape (N, K)
        对硬聚类方法, 返回 one-hot 或距离倒数归一化
```

```
        """
        ...
```

依赖：sklearn.cluster, sklearn.mixture, hdbscan（可选）

---

scx/state/assignment.py — StateAssignment

样本到状态的分配策略，支持硬分配和软分配。

```
class StateAssignment:
    """ 状态分配器：将样本映射到状态（硬/软） """

    @staticmethod
    def hard_assign(
        X_phi: np.ndarray,
        centroids: np.ndarray,
        metric: str = "euclidean"
    ) -> np.ndarray:
        """ 硬分配：每个样本分到最近质心的状态

        Returns: (N,) int 标签
        """
        ...

    @staticmethod
    def soft_assign(
        X_phi: np.ndarray,
        centroids: np.ndarray,
        temperature: float = 1.0,
```

```
        metric: str = "euclidean"
    ) -> np.ndarray:
        """ 软分配：距离倒数 softmax 权重

        Returns: (N, K) float 权重矩阵，每行和为 1
        """
        ...

    @staticmethod
    def soft_assign_gmm(
        X_phi: np.ndarray,
        means: np.ndarray,
        covariances: np.ndarray,
        weights: np.ndarray
    ) -> np.ndarray:
        """GMM 后验概率分配"""
        ...

    @staticmethod
    def state_proportions(assignment: np.ndarray, n_states: int) -> np.ndarray:
        """ 计算每个状态的样本比例 (s)"""
        ...
```

依赖：numpy, scipy.spatial.distance

---

scx/state/metrics.py — StateMetrics

状态质量评估指标。

```python
class StateMetrics:
    """ 状态发现质量评估"""

    @staticmethod
    def silhouette_score(X_phi: np.ndarray, labels: np.ndarray) -> float:
        """ 轮廓系数：评估状态紧致度与分离度"""
        ...

    @staticmethod
    def calinski_harabasz(X_phi: np.ndarray, labels: np.ndarray) -> float:
        """CH 指数：类间离差/类内离差"""
        ...

    @staticmethod
    def davies_bouldin(X_phi: np.ndarray, labels: np.ndarray) -> float:
        """DB 指数：类内相似度/类间分离度（越低越好）"""
        ...

    @staticmethod
    def state_purity(
        labels_pred: np.ndarray,
        labels_true: np.ndarray
    ) -> float:
        """ 状态纯度：预测簇与真实标签的一致性"""
        ...

    @staticmethod
    def state_cohesion(X_phi: np.ndarray, labels: np.ndarray) -> dict:
        """ 每个状态的内聚度（与质心的平均距离）"""
```

```
        ...

    @staticmethod
    def stability_analysis(
        X_phi: np.ndarray,
        method: str = "kmeans",
        n_runs: int = 20,
        k_range: list[int] = None
    ) -> dict:
        """ 稳定性分析：评估不同 k 和初始化的状态一致性"""
        ...
```

依赖：sklearn.metrics

---

scx/expert/ — 专家模块 scx/expert/registry.py — ExpertRegistry

管理多专家系统的注册、查询与维护。

```
class ExpertInfo(NamedTuple):
    name: str                                   # 专家唯一标识名
    predict_fn: Callable                        # f_m: X -> Y 预测函数
    metadata: dict                              # 额外元数据（类型、训练数据量等）

class ExpertRegistry:
    """ 专家注册中心"""

    def __init__(self):
        self._experts: dict[str, ExpertInfo] = {}
```

```
    def register(
        self, name: str,
        predict_fn: Callable,
        metadata: dict | None = None
    ) -> None:
        """ 注册一个专家"""
        ...

    def unregister(self, name: str) -> None:
        """ 注销一个专家"""
        ...

    def get_expert(self, name: str) -> ExpertInfo:
        """ 获取专家信息"""
        ...

    def list_experts(self) -> list[str]:
        """ 列出所有注册的专家名"""
        ...

    def get_predict_fns(self) -> list[Callable]:
        """ 获取所有专家的预测函数列表"""
        ...

    def evaluate_all(
        self, X: np.ndarray, y: np.ndarray,
        metric: Callable = accuracy_score
    ) -> dict[str, float]:
```

```
        """ 评估所有专家的全局性能"""
        ...

    @property
    def size(self) -> int:
        ...
```

依赖：typing, Callable

---

scx/expert/reliability.py — ExpertReliability

核心模块：估计状态条件下的专家可靠性 R_m(s) 和 SCX_m(s)。

```
@dataclass
class ReliabilityResult:
    """ 可靠性估计结果"""
    risk_matrix: np.ndarray            # shape (M, K) — R_m(s)
    scx_matrix: np.ndarray             # shape (M, K) — SCX_m(s)
    risk_std: np.ndarray               # shape (M, K) —标准差
    n_samples_per_state: np.ndarray    # shape (K,) —每个状态用于估计的样本数

class ExpertReliability:
    """ 状态条件专家可靠性估计"""

    def __init__(
        self,
        experts: ExpertRegistry,
        loss_fn: Callable = None,
        tau: float = 0.1,
```

```
        min_samples: int = 5
    ):
        """
        Parameters
        ----------
        experts : ExpertRegistry
        loss_fn : Callable(y_pred, y_true) -> float
            默认: MSE for regression, 0-1 loss for classification
        tau : float
            SCX 可靠性阈值
        min_samples : int
            状态内最少样本数
        """
        self.experts = experts
        self.loss_fn = loss_fn or self._default_loss_fn
        self.tau = tau
        self.min_samples = min_samples

    def estimate_supervised(
        self,
        X: np.ndarray,
        y: np.ndarray,
        state_assignment: np.ndarray,
        n_states: int
    ) -> ReliabilityResult:
        """ 有标签数据的监督估计

        R_m(s) = (1/|L_s|) * Σ_{(x,y) L_s}  (f_m(x), y)
        SCX_m(s) = (1/|L_s|) * Σ_{(x,y) L_s} 1[ (f_m(x), y) <  ]
```

```
        """
        ...

    def estimate_unsupervised(
        self,
        X: np.ndarray,
        state_assignment: np.ndarray,
        n_states: int,
        n_neighbors: int = 5
    ) -> np.ndarray:
        """ 无标签数据的专家间一致性估计

        利用专家预测的一致性来校验/调整监督估计:
        1. 在每个状态内计算专家预测距离矩阵 D(s)    ^{M×M}
        2. 用图方法检测偏差专家
        3. 返回修正置信度

        Returns: (M, K) 置信度调整因子
        """
        ...

    def estimate_joint(
        self,
        X_labeled: np.ndarray, y_labeled: np.ndarray,
        X_unlabeled: np.ndarray,
        state_assignment_labeled: np.ndarray,
        state_assignment_unlabeled: np.ndarray,
        n_states: int
    ) -> ReliabilityResult:
```

```
        """ 联合估计：监督估计 + 无标签校正"""
        ...

    def compute_scx_from_risk(
        self, risk_matrix: np.ndarray
    ) -> np.ndarray:
        """ 从风险矩阵推导 SCX 可靠性

        SCX_m(s) = exp(-  * R_m(s))  或 (1 - R_m(s)) 归一化
        """
        ...

    @staticmethod
    def _default_loss_fn(y_pred: np.ndarray, y_true: np.ndarray) -> np.ndarray:
        """ 自适应损失函数"""
        ...
```

依赖：numpy, scipy.spatial.distance, sklearn.metrics

---

scx/expert/router.py — ExpertRouter

状态条件专家路由：将样本路由到最合适的专家。

```
class ExpertRouter:
    """ 状态条件专家路由"""

    def __init__(
        self,
        experts: ExpertRegistry,
```

```
        reliability: ExpertReliability,
        alpha: float = 1.0,        # 路由 softmax 温度
        tau_hard: float = 0.7      # 硬路由置信度阈值
    ):
        ...
    def hard_route(
        self,
        x: np.ndarray,
        state_weights: np.ndarray,      # (K,) 软分配权重
        risk_matrix: np.ndarray         # (M, K) 最新的 R_m(s)
    ) -> tuple[int, str]:
        """ 硬路由: 选择最可靠的专家

        m*(x) = arg min_m Σ_s  _s(x) • R_m(s)
        """
        ...

    def soft_weights(
        self,
        x: np.ndarray,
        state_weights: np.ndarray,      # (K,)
        risk_matrix: np.ndarray         # (M, K)
    ) -> np.ndarray:
        """ 软加权: 计算每个专家的权重

        w_m(x)   exp(-  • Σ_s  _s(x) • R_m(s))
        """
        ...
```

```
    def predict(
        self,
        x: np.ndarray,
        state_weights: np.ndarray,
        risk_matrix: np.ndarray,
        mode: str = "soft"              # "soft" | "hard"
    ) -> np.ndarray:
        """ 路由预测

        soft mode: ŷ = Σ_m w_m(x) · f_m(x)
        hard mode: ŷ = f_{m*}(x)
        """
        ...
```

依赖：numpy

---

scx/expert/conflict.py — ExpertConflict

专家间冲突检测与仲裁。

```
@dataclass
class ConflictResult:
    state_id: int
    conflict_score: float               # 0~1, 越高冲突越大
    disagreeing_experts: list[str]      # 分歧专家
    consensus_prediction: np.ndarray    # 仲裁结果
    conflict_type: str                  # "hard" | "disagreement" | "divergence"

class ExpertConflict:
```

```
    """ 专家冲突检测与仲裁"""

    @staticmethod
    def pairwise_disagreement(
        predictions: np.ndarray        # (M, N) 专家预测矩阵
    ) -> np.ndarray:
        """ 计算专家两两分歧矩阵 D    ^{M×M}"""
        ...

    @staticmethod
    def state_conflict_score(
        predictions: np.ndarray,       # (M, N_s) 某状态内专家预测
        method: str = "mad"            # "mad" | "pairwise" | "entropy"
    ) -> float:
        """ 计算单个状态内的专家冲突分数"""
        ...

    @staticmethod
    def arbitrate(
        predictions: np.ndarray,        # (M,)
        risk_s: np.ndarray,             # (M,) 该状态下各专家风险
        method: str = "minimum_risk"
    ) -> tuple[np.ndarray, str]:
        """ 仲裁：选出最可靠的预测

        method:
            "minimum_risk" —选 R_m(s) 最小的专家
            "weighted_vote" —按 exp(- • R_m(s)) 加权投票
            "oracle" —需要外部 oracle 介入
```

```
        """
        ...

    @staticmethod
    def detect_conflict_states(
        predictions: np.ndarray,        # (M, N)
        state_assignment: np.ndarray,   # (N,)
        n_states: int,
        threshold: float = 0.3
    ) -> list[ConflictResult]:
        """ 检测所有状态中的专家冲突"""
        ...
```

依赖：numpy, scipy.stats

---

scx/valuation/ — 数据价值评估模块　scx/valuation/state_value.py — StateValue

核心价值函数 V(s) 的实现。

```
class StateValue:
    """ 状态数据价值 V(s) 计算

    V(s) = r̄(s) • (s) • L(s) • [1 - D(s)] • max_m SCX_m(s)
    """

    def __init__(self):
        self.components: dict[str, np.ndarray] = {}
```

```
    def compute(
        self,
        mean_error: np.ndarray,         # (K,) r̄(s)
        proportion: np.ndarray,         # (K,)  (s)
        learnability: np.ndarray,       # (K,) L(s)
        redundancy: np.ndarray,         # (K,) D(s)
        scx_reliability: np.ndarray     # (M, K) SCX_m(s)
    ) -> np.ndarray:
        """ 计算 V(s) 对所有状态

        Returns: (K,) 价值向量
        """
        max_scx = np.max(scx_reliability, axis=0)  # (K,)
        V = mean_error * proportion * learnability * (1 - redundancy) * max_scx
        self.components = {
            "mean_error": mean_error,
            "proportion": proportion,
            "learnability": learnability,
            "redundancy": redundancy,
            "max_scx": max_scx,
            "value": V
        }
        return V

    def get_components(self) -> dict[str, np.ndarray]:
        """ 返回 V(s) 的各分量用于分析"""
        ...

    def rank_states(self, V: np.ndarray) -> np.ndarray:
```

```
        """ 按价值降序排列状态"""
        ...

    def top_k_states(self, V: np.ndarray, k: int) -> np.ndarray:
        """ 返回价值最高的 k 个状态 ID"""
        ...
```

依赖：numpy

---

scx/valuation/classifier.py — DataClassifier

数据四分类器：将状态分为 Valuable / Redundant / Noisy / ExpertDependent。

```
@dataclass
class ClassificationResult:
    """ 状态分类结果"""
    state_id: int
    category: str                    # "valuable" | "redundant" | "noisy" | "expe
    confidence: float
    reason: str                      # 决策原因

class DataClassifier:
    """ 数据四分类器"""

    def __init__(
        self,
        tau_r: float = 0.3,          # 误差阈值
        tau_rho: float = 0.05,       # 密度阈值
```

```
        tau_L: float = 0.4,          # 可学习性阈值
        tau_D: float = 0.7,          # 冗余度阈值
        tau_conflict: float = 0.3    # 冲突阈值
    ):
        ...

    def classify_state(
        self,
        mean_error: float,           # r̄(s)
        proportion: float,           #  (s)
        learnability: float,         # L(s)
        redundancy: float,           # D(s)
        scx_reliability: np.ndarray, # (M,) 该状态下各专家 SCX
        state_id: int = -1
    ) -> ClassificationResult:
        """ 将单个状态分为四类之一"""

        max_scx = np.max(scx_reliability)
        min_scx = np.min(scx_reliability)
        scx_range = max_scx - min_scx

        # Valuable: 高误差 + 高密度 + 高可学习性 + 低冗余
        if (mean_error > self.tau_r and proportion > self.tau_rho
                and learnability > self.tau_L and redundancy < self.tau_D):
            return ClassificationResult(state_id, "valuable", 0.9,
                f"r={mean_error:.3f}> _r,  ={proportion:.3f}> _ , L={learnability:.

        # Redundant: 低误差 + 高密度 + 高冗余
        if (mean_error < self.tau_r / 2 and proportion > self.tau_rho
```

```
                and redundancy > self.tau_D):
            return ClassificationResult(state_id, "redundant", 0.85,
                f"r={mean_error:.3f}< _r/2, D={redundancy:.3f}> _D")

        # ExpertDependent: 专家间分歧大
        if scx_range > self.tau_conflict:
            return ClassificationResult(state_id, "expert_dependent", min(0.9, sc
                f"SCX range={scx_range:.3f}> _conflict")

        # Noisy: 高误差 + 低密度 + 低可学习性
        if (mean_error > self.tau_r and proportion < self.tau_rho
                and learnability < self.tau_L):
            return ClassificationResult(state_id, "noisy", 0.8,
                f"r={mean_error:.3f}> _r,  ={proportion:.3f}< _ , L={learnability:.

        # 默认: 保守视为 valuable
        return ClassificationResult(state_id, "valuable", 0.5, "default: conserva

    def classify_all_states(
        self,
        mean_errors: np.ndarray,
        proportions: np.ndarray,
        learnabilities: np.ndarray,
        redundancies: np.ndarray,
        scx_reliability: np.ndarray
    ) -> list[ClassificationResult]:
        """ 分类所有状态"""
        ...
```

```
    def summary(self, results: list[ClassificationResult]) -> dict:
        """ 生成分类汇总统计"""
        ...

    def classify_sample(
        self,
        sample_error: float,           # r(x)
        sample_density: float,         #  (s(x))
        sample_consistency: float,     # 标签一致性
    ) -> str:
        """ 样本级分类（补充状态级分类）"""
        ...
```

依赖：numpy, dataclasses

---

scx/valuation/learnability.py — LearnabilityScore

可学习性 L(s) 估计。

```
class LearnabilityScore:
    """ 状态可学习性 L(s) 估计

    L(s) = C(s) • [1 - N(s)]
    其中 C(s) 是状态内一致性，N(s) 是状态噪声分数
    """

    @staticmethod
    def from_label_consistency(
        y: np.ndarray,                 # 状态内标签
```

```
        method: str = "entropy"
    ) -> float:
        """ 从标签一致性估计可学习性

        C(s) = 1 - H(Y|s) / log(|Y|)  归一化条件熵
        """
        ...

    @staticmethod
    def from_expert_consensus(
        predictions: np.ndarray,      # (M, N_s) 各专家预测
        labels: np.ndarray            # (N_s,) 参考标签
    ) -> float:
        """ 从专家共识估计可学习性"""
        ...

    @staticmethod
    def from_knn_consistency(
        X_phi: np.ndarray,            # (N_s, d) 表示空间
        labels: np.ndarray,           # (N_s,) 标签
        n_neighbors: int = 5
    ) -> float:
        """ 从 k-NN 标签一致性估计可学习性

        高一致性 → 高可学习性
        低一致性（近邻标签不同）→ 噪声/低可学习性
        """
        ...
```

```
    @staticmethod
    def estimate_noise_fraction(
        X_phi: np.ndarray,
        y: np.ndarray,
        method: str = "knn"
    ) -> float:
        """ 估计状态内的噪声比例 N(s)"""
        ...
```

依赖：numpy, scipy.stats, sklearn.neighbors

---

scx/valuation/noise_score.py — NoiseScore

样本级噪声分数 N(x)。

```
class NoiseScore:
    """ 样本级噪声估计

    NoiseScore(x_i) = r_i  •  (1/(s_i) + )  •  [1 - C(s_i)]
    """

    @staticmethod
    def estimate(
        errors: np.ndarray,           # (N,) r_i —各样本误差
        state_densities: np.ndarray, # (N,)  (s_i) —所属状态密度
        consistencies: np.ndarray,   # (N,) C(s_i) —状态一致性
        epsilon: float = 1e-8
    ) -> np.ndarray:
        """ 计算噪声分数
```

```
        高误差 + 低密度 + 低一致性 → 高分（更像噪声）
        高误差 + 高密度 + 高一致性 → 低分（更像困难样本）
        """
        ...

    @staticmethod
    def detect_label_noise(
        X_phi: np.ndarray,
        y: np.ndarray,
        n_neighbors: int = 5,
        threshold: float = 0.5
    ) -> np.ndarray:
        """ 检测可能的标签噪声样本

        方法：k-NN 标签不一致检测
        Returns: (N,) bool mask — True 表示疑似噪声
        """
        ...

    @staticmethod
    def detect_outlier_noise(
        X_phi: np.ndarray,
        centroids: np.ndarray,
        radii: np.ndarray,
        assignment: np.ndarray,
        n_std: float = 3.0
    ) -> np.ndarray:
        """ 检测离群噪声（远离所属状态质心的样本）"""
```

```
        ...

    @staticmethod
    def rank_by_noise(
        X_phi: np.ndarray,
        y: np.ndarray,
        errors: np.ndarray,
        state_info: StateSpace
    ) -> np.ndarray:
        """ 按噪声分数降序排列样本"""
        ...
```

依赖：numpy, sklearn.neighbors, scipy.spatial

---

scx/valuation/redundancy.py — RedundancyScore

冗余度 D(s) 估计。

```
class RedundancyScore:
    """ 状态冗余度 D(s) 评估

    D(s) 反映状态 s 已被充分覆盖的程度:
    - 高 D(s) → 该状态样本密集、信息增益小
    - 低 D(s) → 该状态有探索空间
    """

    @staticmethod
    def from_coverage(
        X_phi: np.ndarray,           # (N_s, d) 状态内表示
```

```
        centroid: np.ndarray,        # (d,) 状态质心
        radius: float
    ) -> float:
        """ 从覆盖密度估计冗余度

        D(s) = min(1, coverage_volume / total_volume)
        """
        ...

    @staticmethod
    def from_nearest_neighbor_distance(
        X_phi: np.ndarray,
        n_neighbors: int = 3
    ) -> float:
        """ 从平均近邻距离估计冗余度

        平均近邻距离小 → 样本密集 → 高冗余
        D(s) = 1 - tanh(mean_knn_dist / scale)
        """
        ...

    @staticmethod
    def from_information_gain(
        X_phi: np.ndarray,
        y: np.ndarray,
        model: Callable = None
    ) -> float:
        """ 从信息增益估计冗余度
```

```
        去除部分样本后模型性能变化不大 → 高冗余
        """
        ...

    @staticmethod
    def from_diversity(
        X_phi: np.ndarray,
        method: str = "determinantal_point_process"
    ) -> float:
        """ 从多样性估计冗余度

        D(s) = 1 - diversity(s)
        用 DPP 或特征行列式衡量多样性
        """
        ...
```

依赖：numpy, sklearn.neighbors, scipy.linalg

---

scx/action/ — 动作决策模块　scx/action/policy.py — ActionPolicy

状态级动作决策引擎。

```
class ActionPolicy:
    """ 动作决策引擎

    决定每个状态应该执行什么动作:
    a*(s) = arg max_a U(a, s)
```

```
    动作空间: {acquire, relabel, downweight, discard, route}
    """

    def __init__(self, config: dict = None):
        self.config = config or {}

    def decide(
        self,
        classification: ClassificationResult,
        state_value: float,
        budget_remaining: float,
        context: dict = None
    ) -> str:
        """ 根据分类结果和上下文决定单个状态的动作"""
        action_map = {
            "valuable": "acquire",
            "redundant": "skip",
            "noisy": "downweight",
            "expert_dependent": "route"
        }
        base_action = action_map.get(classification.category, "acquire")
        # 考虑 budget 约束和上下文细化
        return self._refine_action(base_action, state_value, budget_remaining, co

    def decide_batch(
        self,
        classifications: list[ClassificationResult],
        state_values: np.ndarray,
        budget: float,
```

```
        context: dict = None
    ) -> dict[int, str]:
        """ 批量决策所有状态

        考虑全局 budget 约束和状态间优先级
        Returns: {state_id: action}
        """
        ...

    def _refine_action(
        self, action: str,
        state_value: float,
        budget_remaining: float,
        context: dict
    ) -> str:
        """ 细化动作：根据实际情况调整"""
        ...
```

依赖：dataclasses, numpy

---

scx/action/acquisition.py — AcquisitionStrategy

状态级主动采集策略。

```
class AcquisitionStrategy:
    """ 状态级数据采集策略

    不再是 point-wise 的 uncertainty sampling,
    而是先选高价值状态，再在状态内选代表点。
```

```
    """

    @staticmethod
    def proportional_allocation(
        state_values: np.ndarray,    # (K,) V(s)
        budget: int
    ) -> np.ndarray:
        """ 按价值比例分配预算给各状态

        B_s = B * V(s) / Σ V(s')
        Returns: (K,) 各状态的采集配额
        """
        ...

    @staticmethod
    def threshold_allocation(
        state_values: np.ndarray,
        budget: int,
        percentile: float = 70.0
    ) -> np.ndarray:
        """ 只对 top 百分位的状态分配预算"""
        ...

    @staticmethod
    def hybrid_allocation(
        state_values: np.ndarray,
        budget: int,
        percentile: float = 60.0,
        min_alloc: int = 1
```

```
    ) -> np.ndarray:
        """ 混合策略：主预算给高价值状态，保底给其他状态"""
        ...

    @staticmethod
    def select_samples_in_state(
        X_phi: np.ndarray,           # (N_s, d) 状态内表示
        X_original: np.ndarray,      # (N_s, d_x) 原始特征
        n_select: int,
        strategy: str = "diverse"    # "random" | "uncertainty" | "diverse" | "co
    ) -> np.ndarray:
        """ 在某状态内选择 n_select 个样本

        "diverse" —基于 k-center 多样性选择
        "uncertainty" —选当前模型最不确定的
        "coreset" —选能覆盖状态空间的代表点
        """
        ...

    @staticmethod
    def state_uncertainty_score(
        X_phi: np.ndarray,
        model_probs: np.ndarray,
        method: str = "margin"
    ) -> np.ndarray:
        """ 计算状态内每个样本的不确定性分数"""
        ...
```

依赖：numpy, sklearn.metrics.pairwise

---

scx/action/compress.py — CompressStrategy

冗余压缩策略（加权 coreset）。

```python
class CompressStrategy:
    """ 状态内冗余压缩

    对冗余状态，选择代表性样本子集并赋予权重。
    """

    @staticmethod
    def weighted_random(
        X_phi: np.ndarray,
        y: np.ndarray,
        weights: np.ndarray,
        keep_ratio: float
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        """ 加权随机采样"""
        ...

    @staticmethod
    def kcenter_greedy(
        X_phi: np.ndarray,
        y: np.ndarray,
        weights: np.ndarray,
        n_keep: int
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        """k-center 贪心选择：最大化覆盖"""
```

```
        ...

    @staticmethod
    def herding(
        X_phi: np.ndarray,
        y: np.ndarray,
        n_keep: int
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        """Herding 选择：保持特征均值"""
        ...

    @staticmethod
    def weighted_coreset(
        X_phi: np.ndarray,
        y: np.ndarray,
        redundancy_scores: np.ndarray,
        keep_ratio: float = 0.5,
        method: str = "weighted_random"
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        """ 主入口：SCX-Compress

        1. 从冗余度计算采样权重 w_i = softmax(-  •  D_i)
        2. 按 keep_ratio 选择 n_keep 个样本
        3. 返回 (X_selected, y_selected, sample_weights)

        Returns
        -------
        X_sub : np.ndarray  选择后的特征
        y_sub : np.ndarray  选择后的标签
```

```
        weights : np.ndarray  样本权重
        """
        ...
```

依赖：numpy, sklearn.metrics.pairwise

---

scx/core/ — 核心框架模块　scx/core/framework.py — SCXFramework

SCX 框架的顶层统一接口。

```
class SCXFramework:
    """SCX 框架主入口

    整合状态发现、专家可靠性、数据价值评估、动作决策的完整流水线。
    """

    def __init__(self, config: "SCXConfig"):
        self.config = config

        # 子模块（在 fit/run 时实例化）
        self.state_discovery: StateDiscovery | None = None
        self.state_space: StateSpace | None = None
        self.expert_registry: ExpertRegistry | None = None
        self.expert_reliability: ExpertReliability | None = None
        self.expert_router: ExpertRouter | None = None
        self.data_classifier: DataClassifier | None = None
        self.state_value: StateValue | None = None
        self.action_policy: ActionPolicy | None = None
```

```
        # 运行状态
        self.history: list[dict] = []
        self.step_count: int = 0

    def fit(
        self,
        X_labeled: np.ndarray,
        y_labeled: np.ndarray,
        X_unlabeled: np.ndarray | None = None,
        experts: ExpertRegistry | None = None,
        phi_fn: Callable | None = None
    ) -> "SCXFramework":
        """ 拟合 SCX 框架

        Pipeline:
        1. 计算 (X) 表示
        2. StateDiscovery.fit(X_phi)
        3. StateSpace ← 构建状态空间
        4. ExpertReliability.estimate_joint(L, U)
        5. 计算 L(s), D(s), V(s), 四分类
        """
        ...

    def step(
        self,
        budget: int = 10,
        context: dict = None
    ) -> dict:
```

```
        """ 执行一步 SCX 主动学习循环

        1. 动作决策 → 每个状态的动作
        2. 执行采集/路由/降权/丢弃
        3. 更新专家可靠性估计
        4. 更新状态价值
        5. 记录历史

        Returns: dict 本轮执行报告
        """
        ...

    def run_pipeline(
        self,
        X_labeled: np.ndarray,
        y_labeled: np.ndarray,
        X_unlabeled: np.ndarray,
        experts: ExpertRegistry,
        n_steps: int = 10,
        budget_per_step: int = 10,
        phi_fn: Callable = None
    ) -> dict:
        """ 运行完整 SCX 流水线

        Returns: 包含完整历史、最终状态、专家可靠性的报告
        """
        ...

    def evaluate(self, X_test: np.ndarray, y_test: np.ndarray) -> dict:
```

```
        """ 评估当前框架性能"""
        ...

    def get_report(self) -> dict:
        """ 生成完整运行报告"""
        ...

    def save(self, path: str) -> None:
        """ 序列化保存"""
        ...

    @classmethod
    def load(cls, path: str) -> "SCXFramework":
        """ 从文件加载"""
        ...
```

依赖：所有子模块

---

scx/core/config.py — SCXConfig

配置管理与校验。

```
@dataclass
class SCXConfig:
    """SCX 框架配置

    提供合理的默认值，支持从 yaml/json 加载。
    """
    # 状态发现
```

```
    n_states: int = 10
    state_discovery_method: str = "kmeans"
    state_discovery_kwargs: dict = field(default_factory=dict)

    # 表示映射
    phi_method: str = "identity"   # "identity" | "pca" | "autoencoder"
    phi_kwargs: dict = field(default_factory=dict)

    # 专家可靠性
    tau: float = 0.1               # SCX 可靠性阈值
    min_samples_per_state: int = 5
    loss_function: str = "auto"    # "auto" | "mse" | "01"

    # 数据四分类
    tau_r: float = 0.3             # 误差阈值
    tau_rho: float = 0.05          # 密度阈值
    tau_L: float = 0.4             # 可学习性阈值
    tau_D: float = 0.7             # 冗余度阈值
    tau_conflict: float = 0.3      # 冲突阈值

    # 动作决策
    default_action: str = "acquire"
    acquisition_strategy: str = "hybrid"   # "proportional" | "threshold" | "hybr
    compress_method: str = "weighted_random"
    compress_keep_ratio: float = 0.5

    # 路由
    routing_mode: str = "soft"        # "soft" | "hard"
    routing_alpha: float = 1.0
```

```
    routing_tau_hard: float = 0.7

    def validate(self) -> bool:
        """ 校验配置合法性"""
        ...

    def to_dict(self) -> dict:
        ...

    @classmethod
    def from_dict(cls, d: dict) -> "SCXConfig":
        ...

    @classmethod
    def from_yaml(cls, path: str) -> "SCXConfig":
        ...

    def save(self, path: str) -> None:
        ...
```

依赖：dataclasses, pyyaml（可选）

---

scx/core/metrics.py — SCXMetrics

SCX 框架整体评估指标。

```
class SCXMetrics:
    """SCX 框架的端到端评估指标"""
```

```
    @staticmethod
    def routing_accuracy(
        X: np.ndarray, y: np.ndarray,
        state_space: StateSpace,
        experts: ExpertRegistry,
        risk_matrix: np.ndarray
    ) -> float:
        """ 路由精度：分配给最优专家的比例"""
        ...

    @staticmethod
    def valuation_correlation(
        V_estimated: np.ndarray,
        V_true: np.ndarray
    ) -> dict:
        """ 价值估计的准确性

        Returns: {"spearman":  , "pearson": r, "kendall":  }
        """
        ...

    @staticmethod
    def acquisition_efficiency(
        history: list[dict],
        metric: str = "auc"
    ) -> float:
        """ 采集效率：budget vs 性能提升曲线下面积"""
        ...
```

```
    @staticmethod
    def compression_retention(
        original_perf: float,
        compressed_perf: float
    ) -> float:
        """ 压缩保留率：压缩后性能/原始性能"""
        ...

    @staticmethod
    def classification_confusion(
        true_categories: np.ndarray,
        predicted_categories: np.ndarray
    ) -> dict:
        """ 分类混淆矩阵"""
        ...

    @staticmethod
    def state_valuation_improvement(
        before: dict, after: dict
    ) -> dict:
        """ 度量 SCX 迭代带来的改进"""
        ...
```

依赖：numpy, scipy.stats, sklearn.metrics

---

scx/utils/ — 工具模块　scx/utils/visualization.py

```python
class SCXVisualizer:
    """SCX 可视化工具集"""

    @staticmethod
    def plot_state_space(
        X_phi: np.ndarray,
        assignment: np.ndarray,
        centroids: np.ndarray,
        save_path: str = None
    ) -> None:
        """ 用 2D/3D 散点图可视化状态空间（PCA/t-SNE 投影）"""
        ...

    @staticmethod
    def plot_reliability_matrix(
        risk_matrix: np.ndarray,
        expert_names: list[str],
        state_labels: list[str],
        save_path: str = None
    ) -> None:
        """ 热力图展示 R_m(s) 或 SCX_m(s) 矩阵"""
        ...

    @staticmethod
    def plot_state_values(
        state_values: np.ndarray,
        components: dict[str, np.ndarray],
        save_path: str = None
    ) -> None:
```

```
        """ 展示 V(s) 及各分量的贡献（堆叠柱状图）"""
        ...

    @staticmethod
    def plot_decision_boundary(
        X: np.ndarray, y: np.ndarray,
        states: np.ndarray,
        save_path: str = None
    ) -> None:
        """ 合成数据的决策边界可视化"""
        ...

    @staticmethod
    def plot_acquisition_curve(
        history: list[dict],
        save_path: str = None
    ) -> None:
        """ 采集曲线: budget vs 性能"""
        ...

    @staticmethod
    def plot_classification_pie(
        results: list[ClassificationResult],
        save_path: str = None
    ) -> None:
        """ 数据四分类饼图"""
        ...

    @staticmethod
```

```
    def plot_expert_routing(
        X: np.ndarray,
        routes: np.ndarray,          # 每个样本路由到的专家 ID
        states: np.ndarray,
        save_path: str = None
    ) -> None:
        """ 专家路由图：颜色表示不同专家"""
        ...
```

依赖：matplotlib, plotly, seaborn, sklearn.manifold (TSNE/PCA)

---

scx/utils/data_loader.py

```
class SCXDataLoader:
    """ 数据加载工具"""

    @staticmethod
    def make_synthetic(
        n_samples: int = 1000,
        n_states: int = 4,
        n_experts: int = 3,
        noise_ratio: float = 0.1,
        redundancy_ratio: float = 0.2,
        random_state: int = 42
    ) -> dict:
        """ 生成合成数据

        Returns
        -------
```

```
        dict with keys:
            X: (N, 2) 原始特征
            y: (N,)   真实标签
            states: (N,) 真实状态归属
            experts: list of predict_fns
            expert_names: list of str
            phi: (N, d) 表示空间特征
        """
        ...

    @staticmethod
    def make_multi_expert_classification(
        dataset: str = "cifar10",
        n_experts: int = 3,
        architectures: list[str] = None,
        noise_config: dict = None
    ) -> dict:
        """ 构造多专家分类数据集

        用不同架构/初始化训练多个专家，添加 label noise。
        """
        ...

    @staticmethod
    def make_mlip_dataset(
        data_path: str = None,
        descriptor: str = "ace",
        n_configs: int = 1000
    ) -> dict:
```

```
        """ 加载 MLIP 数据集，计算描述符"""
        ...

    @staticmethod
    def train_test_split_by_state(
        X: np.ndarray, y: np.ndarray,
        states: np.ndarray,
        test_ratio: float = 0.2,
        stratify: bool = True
    ) -> tuple:
        """ 按状态分层划分训练/测试集"""
        ...
```

依赖：numpy, sklearn.datasets, torchvision（可选）

---

scx/utils/evaluation.py

```
class SCXEvaluation:
    """SCX 评估辅助函数"""

    @staticmethod
    def run_baseline_comparison(
        X: np.ndarray, y: np.ndarray,
        methods: list[str] = ["random", "uncertainty", "coreset", "scx"],
        budget: int = 100,
        n_trials: int = 5
    ) -> pd.DataFrame:
        """ 运行 baseline 比较实验
```

```
        Returns: DataFrame 包含各方法的性能曲线
        """
        ...

    @staticmethod
    def bootstrap_confidence(
        scores: np.ndarray,
        n_bootstrap: int = 1000,
        ci: float = 0.95
    ) -> tuple[float, float, float]:
        """Bootstrap 置信区间估计"""
        ...

    @staticmethod
    def effect_size(
        method_a: np.ndarray,
        method_b: np.ndarray
    ) -> float:
        """Cohen's d 效应量"""
        ...
```

依赖：numpy, pandas, scipy.stats

1.3 模块依赖关系图

```
SCXFramework
  SCXConfig          (无依赖, 纯数据)
  StateDiscovery       sklearn
```

```
    StateSpace       numpy/pandas
  ExpertRegistry     （无依赖）
  ExpertReliability    ExpertRegistry, StateAssignment
    DataClassifier   numpy（阈值判断）
    StateValue       numpy
    LearnabilityScore  sklearn/neighbors
    NoiseScore       sklearn/neighbors
    RedundancyScore   sklearn/neighbors
  ExpertRouter       ExpertRegistry, ExpertReliability
  ExpertConflict     numpy
  ActionPolicy       DataClassifier, StateValue
    AcquisitionStrategy  numpy/sklearn
    CompressStrategy   numpy/sklearn
  SCXMetrics         scipy/sklearn
  Utils
    SCXVisualizer    matplotlib/plotly/sklearn
    SCXDataLoader    sklearn/torchvision
    SCXEvaluation    pandas/scipy
```

---

2. 关键算法伪代码

Algorithm 1: State Discovery — 从原始数据到状态空间

```
Algorithm: StateDiscovery

Input:
  - X_original: (N, d_x) 原始输入数据
```

```
  - Φ : X → ℝ^d 表示映射函数
  - method: "kmeans" | "gmm" | "spectral" | "hdbscan"
  - n_states: K (HDBSCAN 自动)
  - optional hyperparameters (per method)

Output:
  - StateSpace: {states: {s_1,...,s_K}, 每个 s_i 有 centroid, radius, count}
  - hard_assignment: (N,) int 每个样本的状态归属
  - soft_assignment: (N, K) float 软分配概率

Procedure:
  Step 1: Representation
    Φ = Φ(X_original)        # (N, d) 表示矩阵

  Step 2: Dimensionality reduction (optional)
    If dim(Φ) > 50:
      Φ_proj = PCA(Φ, n_components=min(50, N-1))
    Else:
      Φ_proj = Φ

  Step 3: Clustering — method dispatch
    Switch method:
      case "kmeans":
        labels, centroids = KMeans(Φ_proj, K, n_init=20)
        soft = softmax(-dist(Φ_proj, centroids) / temperature)

      case "gmm":
        gmm = GMM(Φ_proj, K, covariance_type='full')
        labels = gmm.predict(Φ_proj)
```

```
        soft = gmm.predict_proba(Φ_proj)
        centroids = gmm.means_

      case "spectral":
        adjacency = rbf_kernel(Φ_proj, gamma=1/p)
        labels = SpectralClustering(adjacency, K)
        centroids = [mean(Φ_proj[labels==k]) for k in range(K)]
        soft = softmax(-dist(Φ_proj, centroids) / temperature)

      case "hdbscan":
        clusterer = HDBSCAN(min_cluster_size=10)
        labels = clusterer.fit_predict(Φ_proj)
        # -1 = noise, 其余为有效状态
        unique_labels = set(labels) - {-1}
        K_actual = len(unique_labels)
        centroids = [mean(Φ_proj[labels==k]) for k in unique_labels]
        # 噪声点软分配到最近状态
        soft = softmax(-dist(Φ_proj, centroids) / temperature)

  Step 4: Build StateSpace
    For k = 0 to K-1:
      mask = (labels == k)
      state_info = StateInfo(
        id = k,
        centroid = centroids[k],
        radius = mean(dist(Φ_proj[mask], centroids[k])),
        count = sum(mask),
        proportion = sum(mask) / N
      )
```

```
      state_space.add_state(state_info)

  Step 5: Post-processing
    If method in {"kmeans", "spectral"}:
      # 检查状态稳定性
      stability_scores = bootstrap_stability(Φ_proj, method, K, n_iter=20)
      If mean(stability_scores) < 0.7:
        Warning("State structure unstable, consider reducing K or changing method'

  Return StateSpace, hard_assignment, soft_assignment
```

Algorithm 2: Expert Reliability Estimation (Supervised + Unsupervised)

```
Algorithm: ExpertReliabilityEstimation

Input:
  - L: (N_L, d_x) labeled data
  - y_L: (N_L,) labels
  - U: (N_U, d_x) unlabeled data  (optional, can be empty)
  - experts: {f_1, ..., f_M}
  - state_assignment_L: (N_L,) state IDs for labeled data
  - state_assignment_U: (N_U,) state IDs for unlabeled data (or None)
  - K: number of states
  -  : SCX threshold
  - min_samples: minimum samples per state

Output:
  - R: (M, K) estimated State-Conditioned Expert Risk
```

```
  - SCX: (M, K) estimated SCX Reliability
  - confidence: (M, K) estimation confidence

Procedure:

  PHASE 1: Supervised Estimation (on labeled set L)



  Initialize R = zeros(M, K), SCX = zeros(M, K)
  Initialize count_s = zeros(K)

  For each expert m = 1 to M:
    y_pred_m = f_m(L)                          # 专家 m 对所有标注样本的预测

    For each state k = 0 to K-1:
      mask = (state_assignment_L == k)
      n_k = sum(mask)
      count_s[k] = n_k

      If n_k >= min_samples:
        # Risk: 平均损失
        losses = loss_fn(y_pred_m[mask], y_L[mask])
        R[m, k] = mean(losses)

        # SCX: 可接受预测的比例
        SCX[m, k] = mean(losses <  )
      Else:
        R[m, k] = NaN       # 不足 min_samples, 需要 shrinkage
        SCX[m, k] = NaN
```

```
  PHASE 1b: Shrinkage Estimation (for small states)


  For each state k where count_s[k] < min_samples:
    # 用全局 + 相邻状态的加权估计
    global_R_m = mean(R[m, :])                # 专家 m 的全局平均风险
    n_k = count_s[k]

    # James-Stein style shrinkage
     = n_k / (n_k + min_samples)              # 收缩因子
    R[m, k] =   * R_raw[m, k] + (1- ) * global_R_m
    SCX[m, k] =   * SCX_raw[m, k] + (1- ) * global_SCX_m


  PHASE 2: Unsupervised Cross-Validation (on unlabeled set U, if available)


  If U is not empty:
    For each state k = 0 to K-1:
      mask_U = (state_assignment_U == k)
      n_u = sum(mask_U)

      If n_u < min_samples: continue

      # 收集专家在该状态的预测
      preds = zeros(M, n_u)
      For m = 1 to M:
```

```
      preds[m, :] = f_m(U[mask_U])

    # 计算专家间分歧矩阵 D   ^{M×M}
    D = zeros(M, M)
    For i = 1 to M:
      For j = i+1 to M:
        D[i,j] = D[j,i] = mean(loss_fn(preds[i], preds[j]))

    # 检测偏差专家
    mean_divergence = mean(D, axis=1)      # 每个专家对其他专家的平均分歧
    outlier_experts = where(mean_divergence > mean(mean_divergence) + 2*std(mean

    # 调整 R: 偏差专家的风险上调，共识专家的风险下调
    For m in outlier_experts:
      R[m, k] = min(1.0, R[m, k] * 1.2)   # 上调 20%
      confidence[m, k] *= 0.8


PHASE 3: Confidence Estimation


For each (m, k):
  n = count_s[k]
  If n >= min_samples:
    # Bootstrap 标准误
    bootstrap_std = bootstrap(R[m, k], n_iter=100)
    confidence[m, k] = 1.0 / (1.0 + bootstrap_std)
  Else:
    confidence[m, k] = n / (n + min_samples) * 0.5    # 低置信度
```

```
  Return R, SCX, confidence
```

Algorithm 3: Data Four-Classification

```
Algorithm: DataFourClassification

Input:
  - mean_errors: (K,) r̄(s) for each state
  - proportions: (K,)  (s) for each state
  - learnabilities: (K,) L(s) for each state
  - redundancies: (K,) D(s) for each state
  - scx_matrix: (M, K) SCX_m(s) for all experts and states
  - thresholds: { _r,  _ ,  _L,  _D,  _conflict}
  - X_phi: (N, d) representation (optional, for sample-level refinement)
  - y: (N,) labels (optional, for sample-level refinement)

Output:
  - state_classifications: list of (state_id, category, confidence, reason)
  - sample_noise_flags: (N,) bool  (optional)

Procedure:

  PHASE 1: State-Level Classification


  For each state k = 0 to K-1:
    r̄ = mean_errors[k]
      = proportions[k]
```

```
    L = learnabilities[k]
    D = redundancies[k]
    scx_range = max(scx_matrix[:,k]) - min(scx_matrix[:,k])
    max_scx = max(scx_matrix[:,k])

    # Rule 1: Valuable — 高误差、高密度、高可学习、低冗余
    If r̄ >  _r AND   >  _  AND L >  _L AND D <  _D:
      class = "valuable"
      conf = 0.9
      reason = f"r={r̄:.3f}> _r,  ={ :.3f}> _ , L={L:.3f}> _L, D={D:.3f}< _D"

    # Rule 2: Redundant — 低误差、高密度、高冗余
    Elif r̄ <  _r/2 AND   >  _  AND D >  _D:
      class = "redundant"
      conf = 0.85
      reason = f"r={r̄:.3f}< _r/2, D={D:.3f}> _D"

    # Rule 3: Expert-Dependent — 专家间分歧大
    Elif scx_range >  _conflict:
      class = "expert_dependent"
      conf = min(0.9, scx_range)
      reason = f"SCX range={scx_range:.3f}> _conflict, best={max_scx:.3f}"

    # Rule 4: Noisy — 高误差、低密度、低可学习
    Elif r̄ >  _r AND   <  _  AND L <  _L:
      class = "noisy"
      conf = 0.8
      reason = f"r={r̄:.3f}> _r,  ={ :.3f}< _ , L={L:.3f}< _L"
```

```
    # Default: 保守视为 Valuable
    Else:
      class = "valuable"
      conf = 0.5
      reason = "conservative default"

    Append (k, class, conf, reason) to state_classifications


  PHASE 2: Sample-Level Refinement (optional)


  If X_phi and y are provided:
    For each sample i = 1 to N:
      s_i = state assignment of sample i
      r_i = loss of current model on sample i
       _i = proportion of state s_i

      If  _i <   AND r_i >  _r:      # 极低密度 + 高误差
        # 检查 k-NN 标签一致性
        neighbors = kNN(X_phi, X_phi[i], k=5)
        neighbor_labels = y[neighbors]
        majority_label = mode(neighbor_labels)
        If y[i] != majority_label:
          noise_flags[i] = "label_noise"
        Elif dist(X_phi[i], centroid[s_i]) > 3 * radius[s_i]:
          noise_flags[i] = "outlier_noise"
        Else:
          noise_flags[i] = "hard_sample"
```

```
  PHASE 3: Summary


  summary = {
    "n_valuable": count(class == "valuable"),
    "n_redundant": count(class == "redundant"),
    "n_noisy": count(class == "noisy"),
    "n_expert_dependent": count(class == "expert_dependent"),
    "valuable_samples": sum(proportions[class == "valuable"] * N),
    "redundant_samples": sum(proportions[class == "redundant"] * N),
    "noisy_samples": sum(proportions[class == "noisy"] * N),
    "expert_dependent_samples": sum(proportions[class == "expert_dependent"] * N),
  }

  Return state_classifications, noise_flags (optional), summary
```

Algorithm 4: State-Wise Acquisition

```
Algorithm: StateWiseAcquisition

Input:
  - state_space: StateSpace with K states
  - V: (K,) state values V(s)
  - class: (K,) state categories
  - budget: B (total samples to acquire)
  - strategy: "proportional" | "threshold" | "hybrid"
  - X_phi: (N, d) representation (for within-state selection)
```

```
  - selection_strategy: "random" | "uncertainty" | "diverse" | "coreset"

Output:
  - acquisition_plan: list of (state_id, n_samples, [sample_indices])

Procedure:

  STEP 1: Filter to valuable states only


  valuable_mask = [class[k] == "valuable" for k in range(K)]
  K_v = sum(valuable_mask)
  If K_v == 0:
    # 没有高价值状态，全部均匀分配
    B_s = [B // K] * K
    B_s[0:B % K] += 1                        # 余数分配
    Return plan with B_s


  STEP 2: Budget allocation across states


  V_valuable = V[valuable_mask]
  Switch strategy:
    case "proportional":
      # 按 V(s) 比例分配
      total_V = sum(V_valuable)
      for idx, k in enumerate(where(valuable_mask)[0]):
        B_s[k] = floor(B * V_valuable[idx] / total_V)
```

```
    # 保证每个有价值状态至少 1 个
    B_s[where(valuable_mask)[0]] = max(1, B_s)

  case "threshold":
    # 只给 top 70% 的状态分配
    V_thresh = percentile(V_valuable, 70)
    top_mask = V_valuable >= V_thresh
    n_top = sum(top_mask)
    base = B // n_top
    for idx, k in enumerate(where(valuable_mask)[0]):
      If top_mask[idx]:
        B_s[k] = base
      Else:
        B_s[k] = 0

  case "hybrid":   # RECOMMENDED
    # 70% 预算给 top 60% 的状态（按 V(s) 比例）
    # 30% 预算给剩下的有价值状态（均匀）
    primary_budget = floor(B * 0.7)
    secondary_budget = B - primary_budget

    V_thresh = percentile(V_valuable, 40)   # top 60%
    top_mask = V_valuable >= V_thresh
    top_indices = where(valuable_mask)[0][top_mask]
    bottom_indices = where(valuable_mask)[0][~top_mask]

    n_top = len(top_indices)
    n_bottom = len(bottom_indices)
```

```
    If n_top > 0:
      total_V_top = sum(V[top_indices])
      for k in top_indices:
        B_s[k] = max(1, floor(primary_budget * V[k] / total_V_top))

    If n_bottom > 0:
      per_state = secondary_budget // n_bottom
      for k in bottom_indices:
        B_s[k] = per_state


STEP 3: Within-state sample selection


For each state k with B_s[k] > 0:
  state_indices = where(state_assignment == k)[0]
  X_state = X_phi[state_indices]

  n_select = min(B_s[k], len(state_indices))

  Switch selection_strategy:
    case "random":
      selected = random_choice(state_indices, n_select, replace=False)

    case "uncertainty":
      # 选当前模型最不确定的样本
      uncertainties = model_uncertainty(X_state)
      top_uncertain = argsort(uncertainties, descending=True)[:n_select]
      selected = state_indices[top_uncertain]
```

```
    case "diverse":        # RECOMMENDED
      # k-center 多样性选择
      selected_indices = kcenter_greedy(X_state, n_select)
      selected = state_indices[selected_indices]

    case "coreset":
      # 自适应密度采样：低密度区多采
      densities = estimate_density(X_state)
      weights = 1.0 / (densities + )
      weights = weights / sum(weights)
      selected = weighted_sample(state_indices, weights, n_select, replace=False

  acquisition_plan.append((k, n_select, selected))

Return acquisition_plan


STEP 4: Budget check


total_allocated = sum(plan.n_samples)
If total_allocated < B:
  # 剩余预算分配给 V(s) 最高尚未饱和的状态
  surplus = B - total_allocated
  Allocate surplus to top V(s) states
```

Algorithm 5: SCX-Compress — Weighted Coreset

```
Algorithm: SCXCompress

Input:
  - D_s: {(x_i, y_i)}_{i=1}^{N_s} dataset for one state s
  -  (D_s): (N_s, d) representation
  - redundancy_scores: (N_s,) per-sample redundancy score D_i
  - keep_ratio:    (0, 1) compression ratio
  - method: "weighted_random" | "kcenter" | "herding"
  -  : temperature for weight softmax (default 1.0)

Output:
  - C_s: compressed dataset {(x_j, y_j, w_j)}_{j=1}^{n_keep}

Procedure:

  STEP 1: Determine keep count


  n_keep = max(1, floor(  * N_s))


  STEP 2: Compute sample weights from redundancy


  # 低冗余 → 高权重 (信息量大)
  # 高冗余 → 低权重 (信息量小)
  weights = softmax(-  * redundancy_scores)
```

```
  # 等价于: w_i = exp(-  * D_i) / Σ_j exp(-  * D_j)
  #   控制权重分布的锐度:   大 → 更集中在低冗余样本


  STEP 3: Subset selection


  Switch method:
    case "weighted_random":   # FAST, DEFAULT
      C_s = weighted_sample(D_s, weights=weights,
                            size=n_keep, replace=False)
      # 样本权重保持 w_j

    case "kcenter":           # BALANCED COVERAGE
      # 贪心 k-center: 选 n_keep 个中心点, 最大化覆盖
      # 用权重修正: 低冗余区域优先选
      C_s = []
      remaining = D_s
      For t = 1 to n_keep:
        # 选距离已选集合最远的点（加权）
        farthest_score = zeros(len(remaining))
        For i, x_i in enumerate(remaining):
          min_dist_to_selected = min(dist(x_i, C_s)) if C_s else 0
          farthest_score[i] = min_dist_to_selected * weights[i]
        x_star = remaining[argmax(farthest_score)]
        C_s.append(x_star)
        # 保留对应权重

    case "herding":           # MEAN PRESERVATION
```

```
  = mean( (D_s))                          # 全集均值
C_s = []
 _selected = zeros(0, d)                   # 已选集合的表示

For t = 1 to n_keep:
  remaining = D_s \ C_s
  scores = zeros(len(remaining))
  For i, x_i in enumerate(remaining):
     _i =  (x_i)
    # 选能校正均值偏差的点
     _current = mean( _selected) if len(C_s) > 0 else 0
    scores[i] =   - _current, _i
  x_star = remaining[argmax(scores)]
  C_s.append(x_star)
   _selected = stack( _selected, (x_star))


STEP 4: Weight adjustment for selected subset


# 归一化子集权重
total_w = sum(w_j for (_, _, w_j) in C_s)
C_s = [(x, y, w / total_w * n_keep) for (x, y, w) in C_s]
# 保持总权重 = n_keep

Return C_s
```

Algorithm 6: Expert Routing

```
Algorithm: ExpertRouting

Input:
  - x: single sample to route
  - experts: {f_1, ..., f_M}
  - state_space: StateSpace with K states
  - R: (M, K) estimated state-conditioned risks
  -  : temperature for weight softmax (default 1.0)
  - mode: "hard" | "soft"
  -  _hard: confidence threshold for hard routing (default 0.7)

Output:
  - ŷ: predicted label
  - route_info: {
      expert_weights: (M,) if soft mode, or
      chosen_expert: int if hard mode,
      dominant_state: int,
      confidence: float
    }

Procedure:

  STEP 1: Compute state assignment for x


   _x =  (x)                                    # Compute representation
  distances = dist( _x, state_space.centroids)  # (K,) distances to centroids
```

```
  = softmax(-distances / temperature)         # (K,) soft assignment weights

dominant_state = argmax( )
state_confidence = max( )


STEP 2: Compute expert weights


# For each expert m, compute weighted risk
expert_scores = zeros(M)
For m = 1 to M:
  expert_scores[m] = Σ_{k=1}^{K}  [k] * R[m, k]    # Expected risk for expert m

# Convert to weights: lower risk → higher weight
expert_weights = softmax(-  * expert_scores)          # (M,)


STEP 3: Route decision


Switch mode:
  case "hard":
    If state_confidence >=  _hard:
      # 状态明确 → 选该状态下最优专家
      m_star = argmin_m R[m, dominant_state]
      ŷ = f_{m_star}(x)
      route_info = {
        "chosen_expert": m_star,
```

```
        "dominant_state": dominant_state,
        "state_confidence": state_confidence,
        "method": "hard_state_route"
      }
    Else:
      # 状态模糊 → 用软加权决策（fallback）
      ŷ = Σ_{m=1}^{M} expert_weights[m] * f_m(x)
      route_info = {
        "expert_weights": expert_weights,
        "method": "soft_fallback"
      }

  case "soft":                                    # RECOMMENDED
    # 加权集成：每个专家的权重乘以其预测
    predictions = [f_1(x), f_2(x), ..., f_M(x)]

    If task is classification:
      # 加权投票
      ŷ = argmax_c Σ_{m=1}^{M} expert_weights[m] * 1[f_m(x) == c]
    Elif task is regression:
      # 加权平均
      ŷ = Σ_{m=1}^{M} expert_weights[m] * f_m(x)

    route_info = {
      "expert_weights": expert_weights,
      "dominant_state": dominant_state,
      "dominant_expert": argmax(expert_weights),
      "method": "soft_weighted_ensemble"
    }
```

```
  STEP 4: Optional — ExpertConflict handling


  # 检查是否存在严重冲突
  If max(expert_weights) / min(expert_weights) < 2:     # 权重分布平坦
    conflict_flag = True
    # 所有专家预测差异大 → 标记需要外部仲裁
    route_info["conflict_flag"] = True
    route_info["predictions"] = [f_1(x), ..., f_M(x)]

  Return ŷ, route_info
```

---

3. 合成实验设计方案

3.1 实验目标

验证 SCX 框架在可控制环境下的五个核心能力：1. 状态发现是否准确恢复预设状态结构 2. 专家可靠性估计 R_m(s) 是否捕获专家在不同状态的行为差异 3. 数据四分类是否正确区分 valuable / redundant / noisy / expert-dependent 4. 状态级主动采集是否优于 point-wise baseline 5. SCX-Compress 是否在保持性能的同时有效压缩数据

3.2 数据生成器设计

3.2.1 状态结构 在 2D 平面上生成 4 个高斯状态：

```
states_config = {
    "s0": {  # 左下 —高密度，线性可分
        "centroid": [2, 2], "cov": [[0.8, 0.2], [0.2, 0.8]], "n": 300, "label_fn"
    },
    "s1": {  # 右上 —高密度，非线性
        "centroid": [7, 7], "cov": [[1.0, -0.3], [-0.3, 1.0]], "n": 300, "label_fn
    },
    "s2": {  # 右下 —中密度，噪声
        "centroid": [7, 2], "cov": [[0.5, 0], [0, 0.5]], "n": 200, "label_fn": "x
        "noise_ratio": 0.3    # 30% 标签翻转
    },
    "s3": {  # 左上 —低密度，最难
        "centroid": [2, 7], "cov": [[0.3, 0.1], [0.1, 0.3]], "n": 100, "label_fn"
    }
}
```

总样本数: 900 (300+300+200+100), 2D 二元分类

3.2.2 专家设计 3 个专家，每个在不同状态上具有不同强项：

专家	方法	s0 精度	s1 精度	s2 精度	s3 精度	设计意图
f_A	Logistic Regression	92%	78%	85%	65%	线性的好，非线性差
f_B	Random Forest (depth=3)	85%	90%	70%	80%	非线性好，噪声敏感

专家	方法	s0 精度	s1 精度	s2 精度	s3 精度	设计意图
f_C	k-NN (k=15)	88%	85%	75%	75%	均衡中庸

3.2.3 冗余注入 对每个状态，随机选择 20% 的样本，对其特征添加小噪声生成” 冗余副本”：

```
def inject_redundancy(X, y, ratio=0.2, noise_scale=0.05):
    n_redundant = int(len(X) * ratio)
    indices = np.random.choice(len(X), n_redundant, replace=False)
    X_dup = X[indices] + np.random.normal(0, noise_scale, (n_redundant, 2))
    return np.vstack([X, X_dup]), np.hstack([y, y[indices]])
```

3.2.4 表示空间

```
def phi_synthetic(X):
    """2D → 6D: 原始特征 + 多项式 + RBF 扩展"""
    x, y = X[:, 0:1], X[:, 1:2]
    return np.hstack([
        X,                      # 原始: x, y
        x**2, y**2,             # 平方: x², y²
        x * y,                  # 交叉: x*y
        np.exp(-(x**2 + y**2) / 10)  # RBF
    ])
```

3.3 Baseline 对比

Method	描述	缩写
Random	随机采样	RND
Uncertainty	最大熵/最小置信度 margin sampling	US
Coreset	k-center coreset	CS
SCX-StateValue	按 V(s) 状态级采集（本文）	SCX-SV
SCX-FourClass	四分类引导的混合采集	SCX-FC
SCX-Compress	加权 coreset 压缩	SCX-CP

3.4 评估指标

指标	公式	用途
Test Accuracy	Acc(f, X_test, y_test)	最终模型性能
Budget Efficiency	AUC of learning curve	有效利用标注预算
Routing Accuracy	P(m* = arg min R_m(s))	路由是否正确
V(s) Rank Correlation	Spearman(V_est, V_true)	价值估计准确性
Classification F1	Macro F1 on four classes	四分类准确度
Compression Retention	Acc_compressed / Acc_full	压缩保持率
Noise Detection Precision	TP / (TP+FP)	噪声识别精度

3.5 实验轮次

每个实验配置运行 20 轮（不同随机种子），报告均值 ± 标准误。

实验序列：

Exp 1.1: 无噪声，无冗余 — 验证状态发现和 V(s) 基础

Exp 1.2: 有噪声 (10%) — 验证噪声检测

Exp 1.3: 有冗余 (20%) — 验证冗余压缩

Exp 1.4: 噪声 + 冗余 — 完整 pipeline 验证
Exp 1.5: 专家分歧状态 — 验证路由

3.6 可视化要求

Figure 1: 状态空间可视化
(a) 原始 2D 数据分布（颜色 = 状态, 标记 = 类别）
(b) 空间 PCA 投影（颜色 = 状态）
(c) 聚类结果 vs 真实状态（混淆矩阵热力图）

Figure 2: 专家可靠性矩阵
(a) R_m(s) 热力图（3×4）
(b) SCX_m(s) 热力图（3×4）
(c) 误差棒图：每个专家的状态条件精度 vs 全局精度

Figure 3: 数据四分类
(a) 2D 散点图（颜色 = 四分类结果）
(b) 各状态分类结果堆叠柱状图
(c) 噪声/冗余检测的混淆矩阵

Figure 4: V(s) 分量分析
(a) V(s) 各分量堆叠柱状图
(b) V_est vs V_true 散点图 + Spearman

Figure 5: 采集曲线
(a) Budget vs Accuracy 对比（SCX vs Baselines）
(b) 各方法每轮的状态选择统计

Figure 6: 压缩保留率

(a) 从 0.1 到 1.0 的压缩率-性能曲线
(b) SCX-Compress vs Random/Coreset 压缩比较

Figure 7: 消融研究
(a) 去掉各 V(s) 分量的性能影响
(b) 不同状态数的敏感性分析

---

4. 通用 ML 实验设计方案

4.1 实验目标

在标准 ML benchmark（CIFAR-10 / CIFAR-100）上验证 SCX 的通用性：1. 状态发现能否捕获有意义的数据结构（类别语义、难度层级）2. 多专家场景下状态条件可靠性是否优于全局可靠性 3. SCX 驱动的主动学习是否优于主动学习经典方法 4. SCX-Compress 能否在视觉任务中有效压缩训练集

4.2 数据集

数据集	样本数	类别	用途
CIFAR-10	60,000	10	主实验
CIFAR-100	60,000	100	扩展验证
TinyImageNet (可选)	110,000	200	大状态空间验证

预处理：- 输入归一化: mean/std per channel - 数据增强: RandomCrop(32, padding=4) + RandomHorizontalFlip - 表示空间 ：取倒数第二层隐藏层激

活（feature embedding）

4.3 多专家场景构造

4.3.1 专家定义 用不同架构训练多个专家，人为制造”状态条件能力差异”：

```
experts_config = {
    "ResNet18": {
        "model": ResNet18(num_classes=10),
        "train_config": {"epochs": 100, "lr": 0.1, "weight_decay": 5e-4},
        "strong_on": ["airplane", "ship", "truck"],   # 对纹理类擅长
    },
    "WideResNet-16-8": {
        "model": WideResNet(depth=16, widen_factor=8, num_classes=10),
        "train_config": {"epochs": 100, "lr": 0.1, "weight_decay": 5e-4},
        "strong_on": ["cat", "dog", "deer"],           # 对形状类擅长
    },
    "ViT-Tiny": {
        "model": ViT(image_size=32, patch_size=4, num_classes=10),
        "train_config": {"epochs": 150, "lr": 0.01, "weight_decay": 0.1},
        "strong_on": ["bird", "frog", "horse"],        # 对全局结构擅长
    }
}
```

4.3.2 专家差异放大 为了验证 Proposition 1（全局排序不足），人为构造专家的”盲区”：

1. 将 CIFAR-10 类别分为 3 组：G1={0,3,8}, G2={2,5,7}, G3={1,4,6,9}

2. 对 Expert A: 在训练时移除组 G2 的 70% 数据
3. 对 Expert B: 在训练时移除组 G3 的 70% 数据
4. 对 Expert C: 在训练时移除组 G1 的 70% 数据

结果: 每个专家在自己"被移除"的组上精度显著下降
→ 验证 SCX 能否发现这种状态条件差异

4.3.3 标签噪声注入

```python
def inject_class_conditional_noise(y, noise_ratio=0.2, symmetric=True):
    """ 注入标注噪声 """
    y_noisy = y.copy()
    n_noise = int(len(y) * noise_ratio)
    noise_indices = np.random.choice(len(y), n_noise, replace=False)

    for idx in noise_indices:
        if symmetric:
            # 对称噪声: 均匀翻转到其他类别
            candidates = [c for c in range(n_classes) if c != y[idx]]
            y_noisy[idx] = np.random.choice(candidates)
        else:
            # 非对称噪声: 特定翻转到相似类 (e.g., cat dog, truck automobile)
            y_noisy[idx] = asymmetric_flip(y[idx])
    return y_noisy, noise_indices
```

4.3.4 冗余注入

1. 对每个类别, 随机选择 15% 的样本
2. 添加小幅度光度扰动 (brightness ±10%, contrast ±10%)

3. 保持标签不变，加入训练集作为"冗余样本"

验证：SCX-RedundancyScore 是否识别这些样本

4.4 状态空间构造

```
# 用预训练 ResNet18 的 penultimate layer 作为表示空间
phi_fn = lambda x: resnet18_feature(x)   # (N, 512)

# 状态发现
state_discovery = StateDiscovery(method="kmeans", n_states=20)
# 为什么 20? CIFAR-10 有 10 类，期望每个类 1-3 个状态（easy/hard/mixed）

# 验证状态与类别的对齐
state_class_confusion = confusion_matrix(true_labels, state_assignment)
# 理想情况：每个状态主要对应一个类别
```

4.5 验证假设

H1: SCX 状态条件可靠性 > 全局可靠性

```
Experiment 2.1: Reliability Estimation
  1. 训练 3 个专家（带盲区构造）
  2. 用 SCX 估计 R_m(s) 和 SCX_m(s)
  3. 比较：
     - 状态条件路由准确率 vs 全局路由
     - 每个专家在"强状态"vs"弱状态"的 R_m(s) 差异

Expected: 状态条件路由比全局路由高 10-20%
```

H2: SCX 状态价值 > 传统 uncertainty

```
Experiment 2.2: Active Learning Comparison
  1. 初始: 每类 10 个标注样本 (CIFAR-10: 100 初始)
  2. 每轮采集 100 个样本, 共 10 轮 (总 budget = 1000)
  3. Baseline:
     - Random
     - Uncertainty (margin sampling)
     - Coreset (k-center)
     - SCX-StateValue (按 V(s) 分配)
     - SCX-FourClass (四分类引导)
  4. 每轮后在测试集上评估

Expected: SCX 方法在预算有限时 (budget < 500) 优势最明显
```

H3: SCX 四分类正确识别噪声

```
Experiment 2.3: Noise Detection
  1. 注入不同水平的 label noise (10%, 20%, 30%)
  2. 用 SCX 的 NoiseScore 和 DataClassifier 识别噪声样本
  3. 评估: Precision, Recall, F1 on noise detection

Expected: SCX 噪声检测 F1 > 0.7 (依赖状态结构良好)
```

H4: SCX-Compress 有效压缩训练集

```
Experiment 2.4: Data Compression
  1. 全量 CIFAR-10 训练集 (50,000)
  2. 用 SCX-Compress 压缩到 50%, 25%, 10%
  3. 用压缩后数据训练 ResNet18
```

4. 对比: Random 压缩, Coreset 压缩

Expected: SCX-Compress 在 50% 压缩率时保持 97%+ 的原始性能

H5: 消融研究

Experiment 2.5: Ablation
1. V(s) 各分量的贡献:
   - remove $\bar{r}$(s) → 只用 density * learnability
   - remove (s) → 只用 error * learnability
   - remove L(s) → 只用 error * density
   - remove D(s) → 不考虑冗余
2. 状态发现方法:
   - KMeans vs GMM vs Spectral vs HDBSCAN
3. 状态数 K 的敏感性:
   - K = 5, 10, 20, 30, 50

Expected: L(s) 和 D(s) 在噪声场景贡献最大

4.6 Baseline 完整列表

#	Baseline	描述	代码来源
1	Random	随机采集	sklearn
2	Uncertainty (Margin)	最小置信度 margin	modAL / 自实现
3	Uncertainty (Entropy)	最大熵	modAL / 自实现
4	BALD	Bayesian AL by Disagreement	baal
5	Coreset (k-center)	多样本覆盖	自实现
6	Coreset (KCenterGreedy)	贪心 k-center	自实现

#	Baseline	描述	代码来源
7	Learning Loss	学习损失预测 (Yoo & Kweon 2019)	自实现
8	VAAL	Variational Adversarial AL	自实现
9	**SCX-StateValue (Ours)**	V(s) 引导的采集	SCX
10	**SCX-FourClass (Ours)**	四分类引导	SCX

资源限制：如果计算资源有限，实现 #1-#5 和 #9-#10 即可，跳过需要额外训练的 #7-#8。

4.7 实验配置表

硬件：1×GPU (RTX 3060 / 4090)
每个实验约 2-4 小时 (CIFAR-10), 4-8 小时 (CIFAR-100)

实验矩阵：

Exp	数据集	噪声	冗余	专家数	状态数	Budget	轮次
2.1	C10	0%	0%	3	20	—	5
2.2	C10	10%	15%	3	20	1000	20
2.3	C10	20%	15%	3	20	1000	20
2.4	C100	10%	15%	3	50	2000	20
2.5	C10	20%	15%	3	20	500	20

5. MLIP 案例实验设计方案

5.1 实验目标

在材料科学 MLIP（Machine Learning Interatomic Potentials）场景中验证 SCX：1. 状态空间能否对应有物理意义的原子局域环境 2. 不同 MLIP（ACE vs NEP vs MACE）是否在不同原子环境中有强弱之分 3. SCX 数据四分类能否指导 DFT 数据的高效采集 4. 与 EGP Paper 1 的 AlGaN gauge-fixing 结果进行对比

5.2 数据集

5.2.1 数据来源 使用 EGP Paper 1 的 AlGaN 数据集：- 结构类型：wurtzite AlGaN 合金（不同 Al 浓度, 0~100%）- 数据量：~2000-5000 原子构型 - 标签：DFT 总能量 (eV) + 原子力 (eV/Angstrom) - 单元大小：~72-128 原子/构型

5.2.2 描述符计算（ 映射）

```
def compute_ace_descriptors(structures, max_degree=4, cutoff=6.0):
    """ 用 ACE (Atomic Cluster Expansion) 描述符作为  映射

    Parameters
    ----------
    structures : list[ase.Atoms]
    max_degree : int
        ACE 展开的最大阶数
    cutoff : float
        截断半径 (Angstrom)
```

```
    Returns
    -------
    X_phi : np.ndarray, shape (n_atoms, d)
        每个原子的 ACE 描述符向量
    structure_ids : np.ndarray, shape (n_atoms,)
        原子所属的 structure index
    """
    # 使用 pyace 或 pacemaker 计算 ACE 描述符
    # 每个原子得到一个 ~200-500 维的描述符向量
    descriptors = []
    for atoms in structures:
        # ACE 描述符：包含 2-body, 3-body 等径向/角向信息
        desc = pyace.calculate_descriptors(atoms, max_degree, cutoff)
        descriptors.append(desc)
    return np.vstack(descriptors)
```

备选描述符：| 描述符 | 维度 | 特点 | 适用性 | |——|——|——|——| | ACE (Basis) | 200-500 | 完备、旋转不变 | 首选 | | SOAP | 200-1000 | 平滑、对称性好 | 备选 | | MBTR | 100-500 | 多体、物理直观 | 备选 | | Atom-centered symmetry functions | 50-200 | 轻量、Behler 风格 | 快速测试 |

5.3 状态空间定义

目标状态（有物理意义的原子局域环境）：

状态	物理意义	标识方法	预期特征
s_Al-bulk	Al 的体相环境	Al 配位数为 12	ACE: 高对称性
s_Ga-bulk	Ga 的体相环境	Ga 配位数为 12	ACE: 高对称性

状态	物理意义	标识方法	预期特征
s_Al-Ga-mix	Al-Ga 混合环境	近邻既有 Al 又有 Ga	ACE: 多元素
s_surface	表面原子	配位数 < 12	ACE: 低对称性
s_defect	缺陷环境	存在空位/间隙	ACE: 异常值
s_N-rich	富 N 环境	N 过剩	ACE: N 信号强
s_strained	高应变环境	键长偏离 $\pm 10\%+$	ACE: 特征偏移

5.3.1 状态发现策略

```
# 方案 A: 无监督发现
state_discovery = StateDiscovery(method="hdbscan", n_states="auto")
# HDBSCAN 自动决定状态数，噪声点归为 outlier state

# 方案 B: 半监督发现（已知物理状态种子）
kmeans = KMeans(n_clusters=7, init=physical_centroids)
# 用已知物理状态的描述符均值初始化质心

# 方案 C: 层次发现（推荐）
# 先粗分 → 再细分
# Level 1: 配位数分组 (bulk vs surface vs defect)
# Level 2: 元素分组 (Al-rich vs Ga-rich vs mixed)
```

5.4 专家定义

```
experts = {
    "ACE": {
        "model": pacemaker.ACE(
            max_degree=4, cutoff=6.0,
```

```
            n_hidden=20, n_basis=8
        ),
        "description": "Atomic Cluster Expansion — polynomial basis",
        "training": "all AlGaN structures",
        "strength": "overall accuracy, good transferability",
    },
    "NEP": {
        "model": gpumd.NEP(
            version=4, cutoff=6.0,
            num_parameters=4000
        ),
        "description": "Neuroevolution Potential — neural network",
        "training": "all AlGaN structures",
        "strength": "fast, good for disordered systems",
    },
    "MACE": {
        "model": mace.MACE(
            r_max=6.0, num_interactions=3,
            hidden_irreps="128x0e + 128x1o"
        ),
        "description": "MACE — equivariant message passing",
        "training": "all AlGaN structures",
        "strength": "state-of-the-art accuracy, data hungry",
    }
}
```

5.5 验证假设

H1: MLIP 专家在不同原子环境上有显著精度差异

Experiment 3.1: Expert Reliability Map
1. 对 AlGaN 数据集计算 ACE 描述符
2. 用无监督方法发现状态
3. 对每个状态 s，计算每个专家的 R_m(s) (energy MAE + force MAE)
4. 可视化 R_m(s) 矩阵 (3 experts × K states)

Expected:
- ACE 在 bulk 状态最优 (多体展开在有序结构中表现好)
- NEP 在 disordered/mixed 状态有竞争力
- MACE 在 surface/defect 状态最优 (消息传递捕获长程)
→ 验证 Proposition 1: 不存在全局最优专家

H2: SCX 状态价值引导的数据采集优于随机采集

Experiment 3.2: Active Learning for DFT
Setting: 主动选择原子构型进行 DFT 计算

1. 初始训练: 随机选 100 个构型
2. 每轮: 选 50 个新构型，进行 DFT 计算，加入训练集
3. 共 10 轮 (总 budget = 500)
4. Baseline:
   - Random: 随机选构型
   - Uncertainty: 当前 ensemble 方差最大的构型
   - SCX: 状态级采集 (按 V(s) 分配)
   - SCX-Diverse: 状态级采集 + 状态内多样性选择
5. 每轮后在测试集上评估 energy/force MAE

Expected: SCX 方法在 300-500 构型时达到随机 800-1000 构型的精度

H3: SCX 四分类识别低质量数据

```
Experiment 3.3: Data Cleaning
  1. 向训练数据注入:
     - 10% 含结构弛豫未收敛的构型 (noisy)
     - 10% 近重复构型 (redundant)
  2. 运行 SCX 四分类
  3. 评估:
     - NoiseScore 是否与弛豫误差正相关
     - RedundancyScore 是否与构型相似度正相关
     - 去除 noisy/redundant 后模型精度变化

Expected: 去除 SCX-identified noisy data 后 MAE 下降 5-10%
```

H4: 状态条件路由优于全局混合

```
Experiment 3.4: Expert Routing
  Setting: 用 SCX 路由状态给最适合的 expert

  1. 全量数据上估计 R_m(s)
  2. 对测试集每个原子结构:
     a. 硬路由: 选该状态下 R_m(s) 最小的 expert
     b. 软路由: 按 R_m(s) 加权 ensemble
     c. 全局集成: 所有 expert 等权平均
  3. 比较 energy/force MAE

Expected: 状态条件软路由误差最小
```

H5: 连接 EGP Paper 1

```
Experiment 3.5: Gauge Fixing Cross-Validation
  Setting: SCX 作为 EGP gauge-fixing 的理论验证

  1. 使用 EGP Paper 1 中的 "gauge" 数据:
     - 每个 expert 在特定状态上的偏差
     - Gauge fixing 后的修正 expert
  2. 用 SCX 评估 gauge fixing 的效果:
     - 修正前: R_m(s) 在各状态的差异
     - 修正后: R_m'(s) — 差异是否缩小
  3. 验证: gauge fixing 是否等价于 SCX 路由的特例

Expected: SCX 提供了一个理解 gauge fixing 的统一理论视角
```

5.6 评估指标

指标	公式	用途
Energy MAE	mean(	E_pred - E_DFT
Force MAE	mean(	F_pred - F_DFT
Per-state RMSE	RMSE on each state	状态级精度
SCX-Routing Gain	(MAE_global - MAE_routed) / MAE_global	路由收益
Acquisition Efficiency	AUC of learning curve	采集效率
Compression Retention	MAE_compressed / MAE_full	压缩保持率

6. 依赖与环境

6.1 Python 依赖清单

```
# ===== 核心依赖（必须）=====
numpy>=1.24.0                  # 数值计算
scipy>=1.10.0                  # 统计、距离、稀疏矩阵
scikit-learn>=1.3.0            # 聚类、降维、评估指标
pandas>=2.0.0                  # 数据处理与汇总
pyyaml>=6.0                    # 配置管理
typing-extensions>=4.0.0       # 类型注解

# ===== 可视化（实验分析）=====
matplotlib>=3.7.0              # 基础图表
seaborn>=0.12.0                # 统计图表
plotly>=5.15.0                 # 交互式图表
scienceplots>=2.0.0            # 科学出版级样式（可选）

# ===== 深度学习（ML benchmark）=====
torch>=2.0.0                   # PyTorch
torchvision>=0.15.0            # 数据集、模型、变换
timm>=0.9.0                    # PyTorch Image Models (ViT 等)

# ===== 主动学习（Baseline）=====
modAL>=0.4.0                   # Active Learning 框架
baal>=1.1.0                    # Bayesian Active Learning（可选）

# ===== MLIP 案例（可选）=====
pyace>=0.5.0                   # ACE 描述符计算
```

```
mace>=0.3.0                    # MACE 势函数
gpumd>=4.0                     # NEP/GPUMD
ase>=3.22.0                    # 原子模拟环境
matgl>=0.3.0                   # 材料图神经网络（可选）

# ===== 开发与测试 =====
pytest>=7.4.0                  # 测试框架
pytest-cov>=4.1.0              # 测试覆盖率
black>=23.0.0                  # 代码格式化
isort>=5.12.0                  # import 排序
mypy>=1.5.0                    # 类型检查
pre-commit>=3.3.0              # 提交前检查
```

6.2 环境配置

```yaml
# environment.yml
name: scx
channels:
  - pytorch
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - numpy>=1.24.0
  - scipy>=1.10.0
  - scikit-learn>=1.3.0
  - pandas>=2.0.0
  - pyyaml>=6.0
  - matplotlib>=3.7.0
```

```
  - seaborn>=0.12.0
  - plotly>=5.15.0
  - pip
  - pip:
    - modAL>=0.4.0
    - scienceplots>=2.0.0

# 或者用 pip
pip install numpy scipy scikit-learn pandas pyyaml matplotlib seaborn plotly
pip install modAL scienceplots

# ML benchmark 用
pip install torch torchvision timm

# MLIP 案例用
pip install ase pyace mace gpumd
```

6.3 计算资源估计

合成实验

实验	CPU	内存	时间 (每轮)	轮次	总时间
Exp 1.1 (无噪声)	1 core	2 GB	2 min	20	40 min
Exp 1.2 (有噪声)	1 core	2 GB	3 min	20	60 min
Exp 1.3 (有冗余)	1 core	2 GB	3 min	20	60 min
Exp 1.4 (全)	1 core	2 GB	5 min	20	100 min
Exp 1.5 (路由)	1 core	2 GB	3 min	20	60 min
合成小计					**~5 小时**

ML Benchmark

实验	GPU	内存	时间 (每轮)	轮次	总时间
C10 专家训练 (3 models)	1×RTX3060	8 GB	2 h	1	2 h
Exp 2.1 (Reliability)	1×RTX3060	8 GB	1 h	5	5 h
Exp 2.2 (AL, 4 methods)	1×RTX3060	8 GB	3 h	20	60 h
Exp 2.3 (Noise)	1×RTX3060	8 GB	3 h	20	60 h
Exp 2.4 (Compress)	1×RTX3060	8 GB	1 h	20	20 h
Exp 2.5 (Ablation)	1×RTX3060	8 GB	2 h	20	40 h
ML 小计					**~190 h (8 天)**

注意：ML 实验占用时间最长。建议：1. 先在 CIFAR-10 上运行缩减版（3 轮 × 4 methods）2. 使用超算资源并行跑不同实验配置 3. 论文初期先用合成实验完成核心验证，ML 实验作为补充

MLIP 案例

实验	CPU	GPU	内存	时间
ACE 描述符计算	8 cores	—	16 GB	1 h
专家训练 (3 models)	8 cores	1×RTX4090	32 GB	24 h
Exp 3.1 (Reliability)	4 cores	—	16 GB	2 h
Exp 3.2 (AL)	8 cores	1×RTX4090	32 GB	48 h
Exp 3.3 (Cleaning)	4 cores	1×RTX4090	16 GB	8 h
Exp 3.4 (Routing)	4 cores	1×RTX4090	16 GB	4 h
MLIP 小计				**~87 h**

6.4 项目开发阶段与时间线

```
Phase 1: 基础设施 (1-2 周)
```

scx/state/ — 状态发现与分配
scx/expert/ — 专家注册与可靠性
scx/valuation/ — 数据价值评估
scx/action/ — 动作决策
scx/core/ — 框架整合
scx/utils/ — 可视化与数据加载
tests/ — 单元测试

Phase 2: 合成实验 (1 周)
experiments/synthetic/
运行 Exp 1.1-1.5
迭代修改算法
论文 Figure 1-7

Phase 3: ML Benchmark (2-3 周, 可并行)
experiments/ml_benchmarks/
训练 CIFAR-10 专家
运行 Exp 2.1-2.5
论文 Figure 8-12

Phase 4: MLIP Case (1-2 周, 可并行)
experiments/mlip_case/
计算 ACE 描述符
运行 Exp 3.1-3.5
论文 Figure 13-17

Phase 5: 论文撰写 (2 周, 与 Phase 3/4 并行)
paper/paper_text/
理论部分 (Definitions + Propositions)

方法部分 (Algorithm)
实验部分 (Results)
arXiv 投稿

6.5 实验运行脚本示例

```
# 合成实验
python experiments/synthetic/run_all.py \
    --output_dir outputs/synthetic \
    --n_runs 20 \
    --seed 42

# 单个 ML Benchmark 实验
python experiments/ml_benchmarks/run_al.py \
    --dataset cifar10 \
    --method scx_state_value \
    --noise_ratio 0.1 \
    --budget 1000 \
    --gpu 0

# MLIP 案例实验
python experiments/mlip_case/run_routing.py \
    --data_path /path/to/algan/structures.xyz \
    --descriptor ace \
    --output_dir outputs/mlip_case
```

附录：关键符号对照表

符号	含义	维度
X	输入空间	—
Y	标签空间	—
S	状态空间	K 个状态
	表示映射函数	X → ^d
_s(x)	样本 x 对状态 s 的软分配权重	(K,)
f_m	第 m 个专家	X → Y
M	专家总数	—
K	状态总数	—
R_m(s)	专家 m 在状态 s 的条件风险	(M, K)
SCX_m(s)	专家 m 在状态 s 的可靠性	(M, K)
V(s)	状态 s 的数据价值	(K,)
r̄(s)	状态 s 的平均误差	(K,)
(s)	状态 s 的样本比例	(K,)
L(s)	状态 s 的可学习性	(K,)
D(s)	状态 s 的冗余度	(K,)
C(s)	状态 s 的一致性	(K,)
N(x)	样本 x 的噪声分数	(N,)
	SCX 可靠性阈值	标量
	路由 softmax 温度	标量
	权重分布锐度	标量